



山东工商学院
Shandong Technology and Business University

神经网络与深度学习综合实训

项目报告

小组编号: 17

小组成员: 首宇鑫 周大评

指导教师: 邓富文

日期: 2026.06.30

代码仓库地址:

一、项目介绍

本项目基于《神经网络与深度学习综合实训》指导书中的手势数字识别任务完成。指导书要求在初始代码结构基础上补全 `model.py`、`train.py`、`test.py`、`inference.py` 四个文件，实现手势图片分类模型的训练、测试和推理；在扩展部分可以进一步进行多模型性能比较、新模型设计和 GUI 界面开发。

项目首先复现指导书给出的基础流程，使用指导书原始 64 x 64 RGB 数据集训练 CNN 0-5 分类模型。随后针对真实摄像头中背景复杂、手部区域不稳定、原始数据集不包含 6-9 类别等问题，引入中国数字手势数据集，重新构建 0-9 分类数据清单，训练并比较增强 CNN、EfficientNet-B0、ResNet34 等模型。最终项目保留指导书基础版模型，同时将多个扩展模型和逐数字协同评分规则组织为 `GestureDigitRecognizer_model.pt` 统一模型包，在 Web 端提供上传图片识别和摄像头实时识别。

项目最终实现的主要功能如下：

- 支持 0-9 手势数字识别。
- 保留指导书初始代码结构和 # START/# END 补全方式，其中最初始版本为指导书数据集 CNN 0-5。
- 支持 PyTorch 模型训练、测试、单张图片推理和 Web 端推理服务。
- 支持上传图片识别、摄像头实时识别、深色/浅色主题切换。
- 支持 ROI 多候选、手部区域裁剪、背景弱化和模型重排序，降低复杂背景影响。
- 支持多模型对比，并根据逐类表现进行协同评分。

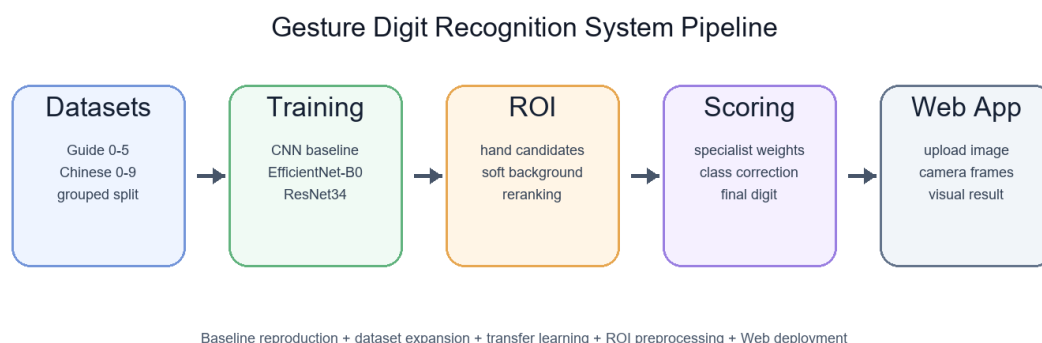


图 1 系统整体流程

从整体流程看，项目不是直接替换指导书任务，而是在“指导书基线复现”的基础上逐步扩展。原始 CNN 0-5 模型用于完成基础实训要求；新增 0-9 数据集和迁移学习模型用于解决类别范围不足和真实场景泛化不足；ROI 与 Web 模块用于解决实际使用中输入画面不稳定、背景干扰强、摄像头实时识别结果跳动等问题。

二、项目实施过程

2.1 完整代码仓库与文件说明

完整代码将随 GitHub 仓库一起提交，仓库地址放置在本报告第二页醒目位置。项目整理后，根目录保留可直接运行的核心代码，实验过程脚本和旧版本尝试放入 实验过程归档/，这样既方便运行当前系统，也便于在报告中说明模型优化过程。

```
gesture_rebuild_from_gitee/
├─ images/                # 指导书数据集索引及新增 chinese/train.txt、test.txt
├─ models/                # 指导书模型、扩展模型和统一模型包
├─ web/                   # Web 前端页面
├─ docs/                  # 指导书、报告模板、报告图片和最终报告
├─ 实验过程归档/          # 数据集构建、扩展训练和评估脚本归档
├─ dataset.py             # 数据集读取
├─ model.py               # 指导书 CNN 模型定义
├─ train.py               # 指导书训练入口
├─ test.py                # 指导书测试入口
├─ inference.py           # 单张图片推理入口
├─ enhanced_model.py      # 扩展 CNN 模型
├─ hand_detector.py       # ROI 候选、手部裁剪和背景弱化
└─ app_server.py          # Web 服务与模型集成推理接口
```

核心代码说明如下：

文件	作用	说明
dataset. py	数据读取	按 train. txt/test. txt 读取图片路径和标签，返回 image 与 label。
model. py	指导书 CNN	在 # START/# END 区域补全 CNN 特征提取器和分类器。
train. py	基础训练	使用交叉熵损失和 Adam 优化器训练指导书 CNN 0-5 模型。
test. py	基础测试	在指导书测试集上统计正确数和准确率。
inference. py	单图推理	读取单张图片并输出预测数字和置信度。
enhanced_model. py	扩展模型	定义增强 CNN，用于新增 0-9 数据集对比。

hand_detector.py	ROI 预处理	构建多个手部候选区域，进行背景弱化和质量评分。
app_server.py	Web 后端	加载统一模型包，提供 /health 和 /api/predict 接口。
web/index.html	Web 前端	实现图片上传、摄像头实时识别和主题切换。

2.2 版本演进说明

为了避免将指导书原始任务和后续扩展混淆，项目按以下版本逐步推进：

版本	数据集	模型/功能	说明
V1 基线版本	指导书原始 0-5	指导书 CNN	完成指导书要求的训练、测试和推理。
V2 数据扩展	中国手势 0-9	增强 CNN	根据文件名前缀补充 6-9 类别。
V3 模型扩展	中国手势分组测试集	EfficientNet-B0、ResNet34	使用高分辨率输入和迁移学习。
V4 应用扩展	Web 图片/摄像头	ROI 与协同评分	面向复杂背景和实时识别做落地优化。

其中 V1 完全基于指导书数据集和指导书 CNN 结构完成，6-9 类别并不是指导书原始数据集提供的，而是在 V2 之后通过新增中国手势数据集补充得到。报告中后续提到的“新增模型”均指 V2/V3 阶段在新增数据集上训练得到的模型，不与指导书原始 CNN 0-5 基线混为一类。

2.3 数据集读取与划分

指导书原始数据集使用文本文件记录样本路径和标签，例如：

```
./images/train/signs_3/img_0000.png 4
./images/train/signs_3/img_0001.png 1
```

在 dataset.py 中，程序逐行读取标注文件，并返回图像张量和整数标签：

```
class CustomDataset(Dataset):
    def __init__(self, annotations_file, img_dir, transform):
        with open(annotations_file, "r") as f:
            self.labels = [line.strip('\n').split(" ") for line in f.readlines()]
        self.img_dir = img_dir
        self.transform = transform()
        self.target_transform = int

    def __getitem__(self, idx):
        image = Image.open(self.labels[idx][0])
        label = self.labels[idx][1]
```

```

if self.transform:
    image = self.transform(image)
if self.target_transform:
    label = self.target_transform(label)
return {'image': image, 'label': label}

```

指导书原始数据集只包含 0-5 六个类别，训练集共 4320 张，测试集共 480 张，各类样本数量均衡，每类训练样本 720 张、测试样本 80 张。该版本对应 models/model_0-5test_own.pkl，是后续所有扩展的基线。

新增中国数字手势数据集目录为：

extra_data/Chinese-number-gestures-recognition/digital_gesture_recognition/resized_img
数据文件名形如：

6_1_0_167.jpg

其中第一个数字 6 表示真实标签。数据集构建脚本从文件名前缀解析标签，并生成指导书风格的 train.txt/test.txt：

```

def scan_images() -> list[tuple[str, int]]:
    samples = []
    for img_path in sorted(SRC_DIR.glob("*.jpg")):
        label = int(img_path.name.split("_", 1)[0])
        rel_path = f"./{REL_PREFIX.as_posix()}/{img_path.name}"
        samples.append((rel_path, label))
    return samples

```

为了避免相似图片泄漏到测试集，新增数据集不是简单按单张图片随机划分，而是按来源组划分。对于 6_1_0_167.jpg 这类文件，去掉最后一个编号作为来源组标识：

```

def source_group_key(rel_path: str, label: int) -> str:
    stem = Path(rel_path).stem
    parts = stem.split("_")
    if len(parts) <= 1:
        return f"{label}:{stem}"
    return f"{label}:{'_' . join(parts[:-1])}"

```

新增数据集共 19383 张图片，生成训练集 15461 张、测试集 3922 张，覆盖 0-9 十个类别。

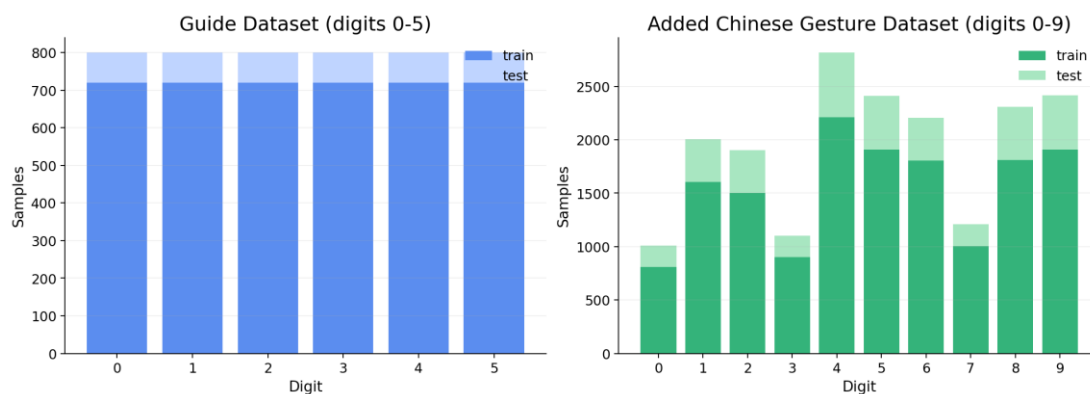


图 2 指导书数据集与新增数据集类别分布

从图 2 可以看出，指导书数据集只包含 0-5，且各类数量均衡；新增中国手势数据集覆盖 0-9，但不同数字样本数并不完全一致，例如 4、5、8、9 样本较多，0、3、7 样本较少。这种分布差异会影响模型逐类准确率，也是后续进行逐类分析和协同权重校正的原因之一。

2.4 指导书 CNN 模型定义

基础模型采用 CNN 结构，由卷积、BatchNorm、ReLU、池化和全连接分类器组成。代码填写在指导书要求的 # START 和 # END 区域之间：

```
self.features = nn.Sequential(
    nn.Conv2d(3, 32, kernel_size=3, padding=1),
    nn.BatchNorm2d(32),
    nn.ReLU(inplace=True),
    nn.MaxPool2d(kernel_size=2, stride=2),
    nn.Conv2d(32, 64, kernel_size=3, padding=1),
    nn.BatchNorm2d(64),
    nn.ReLU(inplace=True),
    nn.MaxPool2d(kernel_size=2, stride=2),
    nn.Conv2d(64, 128, kernel_size=3, padding=1),
    nn.BatchNorm2d(128),
    nn.ReLU(inplace=True),
    nn.MaxPool2d(kernel_size=2, stride=2),
    nn.Conv2d(128, 256, kernel_size=3, padding=1),
    nn.BatchNorm2d(256),
    nn.ReLU(inplace=True),
    nn.AdaptiveAvgPool2d((1, 1)),
)
self.classifier = nn.Sequential(
    nn.Flatten(),
    nn.Dropout(p=0.35),
    nn.Linear(256, 128),
    nn.ReLU(inplace=True),
    nn.Dropout(p=0.2),
    nn.Linear(128, 10),
)
```

前向传播流程如下：

```
def forward(self, x):
    x = self.features(x)
    x = self.classifier(x)
    return x
```

该 CNN 结构保持了指导书代码的轻量特点，训练和推理速度较快。对于原始 64 x 64 图片，卷积层可以提取边缘、轮廓和局部纹理，全连接分类器再输出类别概率。但当输入图像来自高清摄像头时，如果直接缩放到 64 x 64，手指细节和手势边界会明显损失，因此后续又引入了 224 x 224 输入的迁移学习模型进行对比。

2.5 训练、测试与推理流程

训练流程使用交叉熵损失和 Adam 优化器。每个 batch 完成前向传播、损失计算、反向传播和参数更新，并统计训练准确率：

```
for batch, sample in enumerate(dataloader):
    X = sample['image'].to(device)
    y = sample['label'].to(device)

    pred = model(X)
    loss = loss_fn(pred, y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    total_loss += loss.item() * X.size(0)
    correct_num += (pred.argmax(1) == y).type(torch.float).sum().item()
```

训练入口默认使用指导书原始 images/train.txt，模型保存为：

models/model_0-5test_own.pkl

测试时关闭梯度计算，并统计测试集预测正确数量：

```
with torch.no_grad():
    for sample in dataloader:
        X = sample['image'].to(device)
        y = sample['label'].to(device)
        pred = model(X)
        predicted = pred.argmax(1)
        correct_num += (predicted == y).type(torch.float).sum().item()
```

```
accuracy = correct_num / size
print(f"Accuracy: {accuracy:.2%}, Correct: {int(correct_num)}/{size}")
```

单张图片推理时，先读取图片并转换为 Tensor，再通过 Softmax 得到置信度：

```
image = Image.open(image_path).convert("RGB")
image_tensor = torch.unsqueeze(transform(image), 0).to(device)
```

```
with torch.no_grad():
    output = model(image_tensor)
    probability = torch.softmax(output, dim=1)
    confidence, predicted = torch.max(probability, 1)
```

2.6 新增模型训练与选择

为了比较不同模型的泛化能力，项目在 V1 指导书 CNN 基线之外，继续训练了三类新增模型：增强 CNN、EfficientNet-B0 和 ResNet34。其中增强 CNN 沿用了指导书小 CNN 的轻量思路，但训练类别扩展为 0-9；EfficientNet-B0 和 ResNet34 使用 ImageNet 预训练权重，并将分类头改为 10 分类。

主要模型对比结果如下：

模型	输入尺寸	测试方式	测试准确率
指导书 CNN	64 x 64	original 0-5 test	62.29%

增强 CNN	64 x 64	source grouped 0-9 test	56.09%
EfficientNet-B0	224 x 224	source grouped 0-9 test	74.71%
ResNet34	224 x 224	source grouped 0-9 test	84.19%

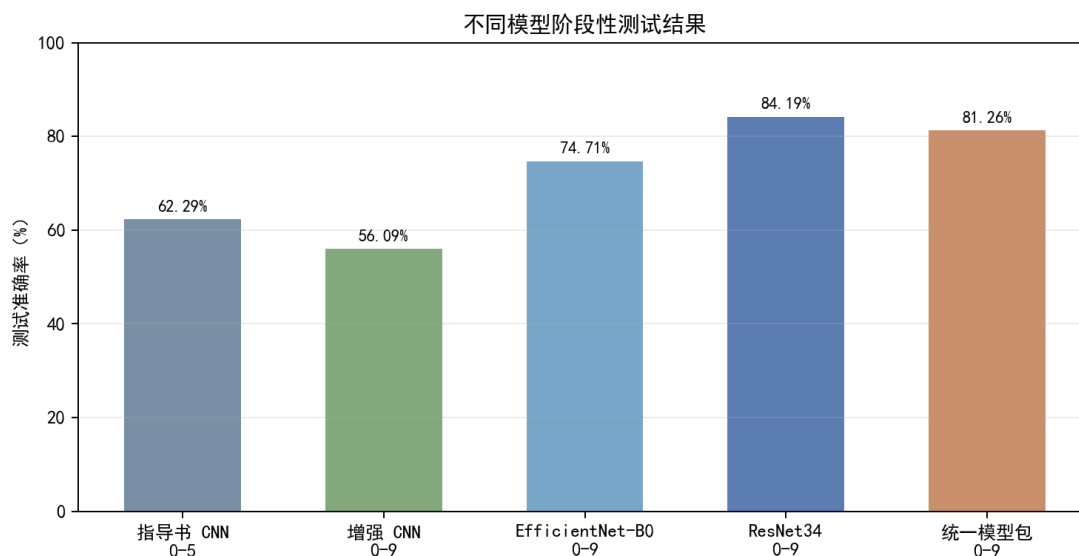


图 3 不同模型阶段性测试结果

需要说明的是，图 3 中指导书 CNN 的测试集是原始 0-5 测试集，而新增模型的测试集是中国手势 0-9 分组测试集，两者难度和类别范围并不完全相同。因此该图主要用于展示实验演进和模型能力变化，不能简单理解为在同一测试集上的横向排名。真正用于 0-9 落地识别的主要依据，是新增数据集上的 EfficientNet-B0、ResNet34 和增强 CNN 的分组测试结果。

ResNet34 的构建方式如下：

```
def build_model(weights=ResNet34_Weights.IMAGENET1K_V1):
    model = resnet34(weights=weights)
    in_features = model.fc.in_features
    model.fc = torch.nn.Linear(in_features, 10)
    return model
```

训练阶段使用数据增强提升复杂场景泛化能力，包括随机裁剪、仿射变换、颜色扰动、灰度化、模糊和随机擦除：

```
transforms.Compose([
    transforms.Resize((288, 288)),
    transforms.RandomResizedCrop(224, scale=(0.58, 1.0), ratio=(0.74, 1.28)),
    transforms.RandomAffine(degrees=20, translate=(0.14, 0.14),
                            scale=(0.78, 1.18), shear=(-7, 7)),
    transforms.ColorJitter(brightness=0.38, contrast=0.38,
                            saturation=0.25, hue=0.025),
    transforms.RandomGrayscale(p=0.05),
    transforms.RandomApply([transforms.GaussianBlur(kernel_size=3)], p=0.25),
    transforms.ToTensor(),
```



```

transforms.RandomErasing(p=0.22, value="random"),
transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
1)

```

数据增强的目的不是简单扩大样本数量，而是模拟真实使用中的手势偏移、尺度变化、光照变化、摄像头轻微模糊和局部遮挡。对于 Web 摄像头实时识别，这类扰动比只在干净样本上训练更接近落地环境。

2.7 ROI 与复杂背景优化

摄像头实时识别时，如果直接把整帧送入分类模型，模型容易识别背景而不是手势。因此项目增加了 `hand_detector.py`，在分类前先生成多个手部 ROI 候选，再根据分类置信度和检测分数进行重排序。

ROI 模块优先尝试 MediaPipe Hands；当环境不支持时，使用 OpenCV 肤色掩膜、形态学处理和中心裁剪兜底。为了避免硬分割失败导致手部被破坏，项目采用“软背景弱化”策略：保留疑似手部区域清晰度，将背景模糊和去饱和。



图 4 ROI 裁剪与背景弱化示意

ROI 处理的核心思路是“尽量把手放大，同时不要过度相信一次分割结果”。如果硬实例分割不准确，手指边缘可能被切掉，反而导致分类模型更难识别；软背景弱化则保留原图中的手部主体，只降低背景对模型的吸引力。

```

def _segment_hand_foreground(crop, source_size=None):
    rgb = np.asarray(crop.convert("RGB"))
    skin = _skin_mask(rgb)
    skin = cv2.morphologyEx(skin, cv2.MORPH_OPEN, kernel, iterations=1)
    skin = cv2.morphologyEx(skin, cv2.MORPH_CLOSE, kernel, iterations=2)

    foreground = cv2.GaussianBlur(foreground, (9, 9), 0)
    alpha = np.clip(foreground.astype(np.float32) / 255.0, 0.0, 1.0)
    blurred = cv2.GaussianBlur(rgb, (0, 0), sigmaX=4.5, sigmaY=4.5)
    composited = rgb * alpha[..., None] + blurred * (1.0 - alpha[..., None])
    return Image.fromarray(composited.astype(np.uint8)), True, coverage

```

2.8 Web 端扩展

Web 端由 app_server.py 提供接口，web/index.html 提供前端页面。服务端接收 base64 图片，经 ROI 预处理、模型推理和协同评分后返回 JSON 结果：

```
def do_POST(self):
    if urlparse(self.path).path != "/api/predict":
        self._send_json(404, {"error": "Not found"})
        return
    length = int(self.headers.get("Content-Length", "0"))
    image, requested_mode = decode_image_from_json(self.rfile.read(length))
    self._send_json(200, predict_from_image(image, requested_mode))
```

Web 前端支持上传图片 and 摄像头实时帧识别，并显示预测数字、置信度、运行设备、FPS、响应时间和 ROI 状态。调试模式 ?debug=1 可以展示各模型单独预测结果，便于分析模型差异。经过多轮界面优化后，页面支持深色/浅色主题切换，并尽量保持开始识别前后布局稳定，避免点击按钮后文本和卡片乱跳。

2.9 统一模型包与协同评分

实际测试中发现，不同模型擅长的数字不同。例如 ResNet34 在整体测试中表现最好，EfficientNet-B0 对 6、7、9 等类别有补充价值，增强 CNN 对 1、3 等部分手势有补充作用。因此最终没有只使用单一模型，而是将多个模型权重、逐数字专家权重和易混类别校正规则组织为 GestureDigitRecognizer_model.pt，作为 Web 端默认调用的统一模型包。

最终模型配置保存在 models/GestureDigitRecognizer_model.json，模型包文件为 models/GestureDigitRecognizer_model.pt。关键配置如下：

```
{
  "name": "GestureDigitRecognizer_model",
  "display_name": "手势数字识别模型",
  "base_models": {
    "guide": "models/model_0-5test_own.pkl",
    "chinese_aug": "models/chinese_cnn_0_9_aug.pkl",
    "efficientnet_b0": "models/chinese_efficientnet_b0_grouped_0_9.pth",
    "resnet34": "models/chinese_resnet34_grouped_0_9.pth"
  },
  "specialist_weight_boosts": {
    "resnet34": {"1": 1.28, "2": 1.24, "4": 1.22, "8": 1.06},
    "chinese_aug": {"1": 2.85, "3": 0.95},
    "efficientnet_b0": {"6": 1.48, "7": 1.24, "9": 1.05}
  }
}
```

协同评分时，各模型输出概率会根据该模型在对应数字上的测试表现进行加权，再对 3、7、8/2 等易混数字做保守校正。该策略本质上不是重新训练一个神经网络，而是将多个已训练模型、逐数字专家权重和混淆校正规则组织成一个统一调用的识别模型包，使 Web 端只需要调用 GestureDigitRecognizer_model.pt 对应的默认入口即可完成最终预测。

三、项目运行结果

3.1 指导书基础模型运行结果

指导书基础 CNN 0-5 模型完成后，模型文件保存为：

models/model_0-5test_own.pkl

该版本只使用指导书原始数据集，不包含新增中国手势数据，也不包含 6-9 类别。在原始测试集上，指导书 CNN 模型可完成 0-5 手势分类，测试命令与输出如下：

```
& C:\Users\Alne\AppData\Local\miniconda3\envs\CV\python.exe test.py
```

```
Accuracy: 62.29%, Correct: 299/480
```

由于原始数据集分辨率较低、背景单一，且类别范围不足，在摄像头真实环境下会出现泛化不足，因此需要继续扩展数据集和模型。

```
Program Test and Inference Output

$ python test.py
Test Error:
Accuracy: 62.29%, Correct: 299/480

$ python inference.py
Image path: ./images/test/signs/img_0006.png
Prediction: 3
Confidence: 100.00%
Output tensor: tensor([[ -10.9500,  12.8658,   1.2219,  23.5180, -22.3723,  -9.9355, -17.2302,
 -14.5248, -13.5864, -13.8276]], device='cuda:0')
```

图 5 程序测试与推理输出

3.2 新增模型对比结果

新增中国手势数据集按来源组切分后，训练集共有 15461 张图片，测试集共有 3922 张图片，覆盖 0-9 十个类别。主要新增模型结果如下：

模型	正确数/测试数	测试准确率	说明
增强 CNN	2200 / 3922	56.09%	小模型速度快，但复杂背景泛化不足。
EfficientNet-B0	2930 / 3922	74.71%	迁移学习模型，部分数字表现突出。
ResNet34	3302 / 3922	84.19%	单模型整体准确率最高，作为主要参考模型。
GestureDigitRecognizer_model.pt	3187 / 3922	81.26%	统一模型包，整合多模型权重、ROI 和校正规则。

ResNet34 逐类准确率如下：

数字	正确数/测试数	准确率
0	108 / 201	53.73%
1	365 / 401	91.02%

2	370 / 401	92.27%
3	170 / 202	84.16%
4	552 / 605	91.24%
5	362 / 503	71.97%
6	257 / 402	63.93%
7	155 / 202	76.73%
8	467 / 501	93.21%
9	496 / 504	98.41%

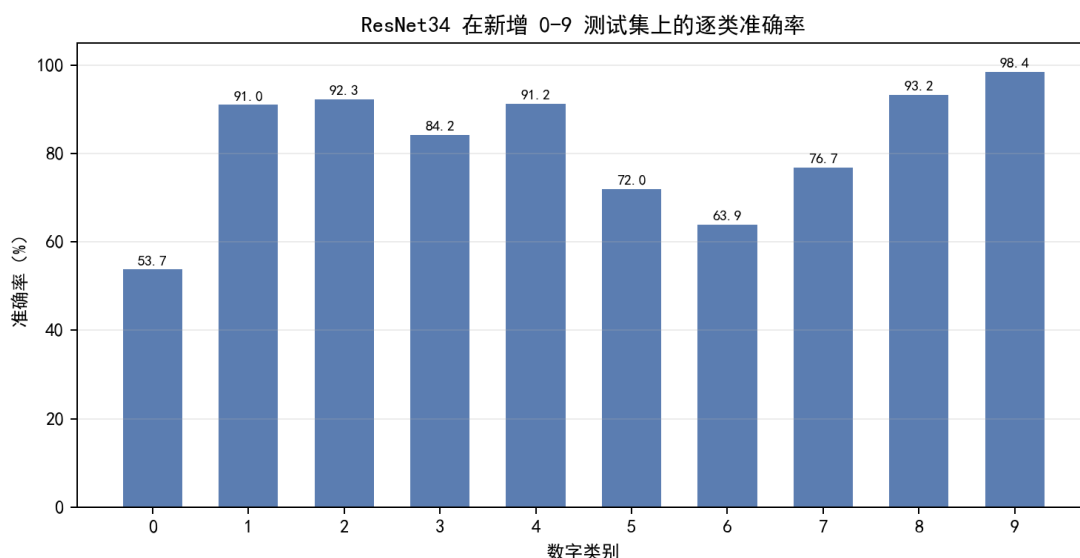


图 6 ResNet34 在新增 0-9 测试集上的逐类准确率

从结果可以看出，高分辨率迁移模型明显优于 64 x 64 小 CNN，但低样本类和容易混淆的手势仍然存在误识别。ResNet34 对 1、2、4、8、9 的识别较稳定，但 0、6 的准确率明显偏低，说明模型对部分手势形态和背景变化仍不够鲁棒。

3.3 统一模型包测试结果

为了验证 Web 默认模型包的综合能力，项目在同一份 images/chinese/test.txt 测试集上对 GestureDigitRecognizer_model.pt 进行了直接输入评估。该测试不引入摄像头 ROI 裁剪，只比较分类层输出，因此可以更清楚地观察协同权重对模型结果的影响。

```
dataset: images/chinese/test.txt
evaluation: direct_model_input_without_roi
samples: 3922
packaged_model: models/GestureDigitRecognizer_model.pt
elapsed_seconds: 96.3
accuracy: 81.26%
correct: 3187/3922
```

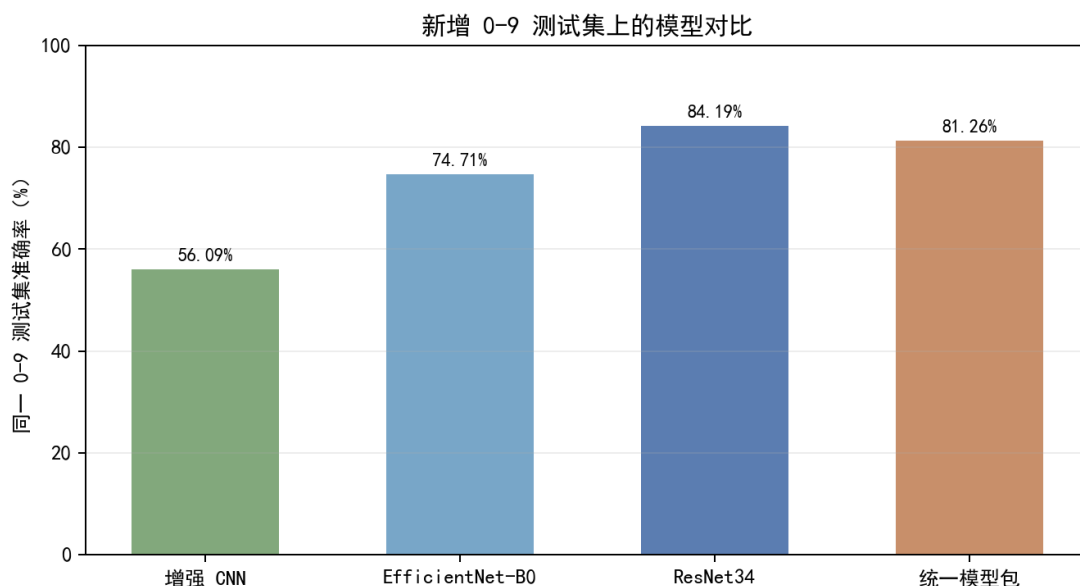


图 7 新增 0-9 测试集上的模型对比

从图 7 可以看出，GestureDigitRecognizer_model.pt 的整体准确率为 81.26%，低于单模型 ResNet34 的 84.19%，但明显高于增强 CNN 和 EfficientNet-B0。该结果说明最终模型包不是为了在每一个静态测试指标上超过 ResNet34，而是面向 Web 端实际使用，把 ResNet34 主干能力、部分模型专家优势、ROI 预处理和易混类别校正统一到一个入口中。

统一模型包与 ResNet34 逐类对比如下：

数字	ResNet34 正确数 / 测试数	ResNet34 准确率	统一模型包正确数 / 测试数	统一模型包准确率
0	108 / 201	53.73%	102 / 201	50.75%
1	365 / 401	91.02%	388 / 401	96.76%
2	370 / 401	92.27%	346 / 401	86.28%
3	170 / 202	84.16%	186 / 202	92.08%
4	552 / 605	91.24%	552 / 605	91.24%
5	362 / 503	71.97%	316 / 503	62.82%
6	257 / 402	63.93%	213 / 402	52.99%
7	155 / 202	76.73%	154 / 202	76.24%
8	467 / 501	93.21%	431 / 501	86.03%
9	496 / 504	98.41%	499 / 504	99.01%

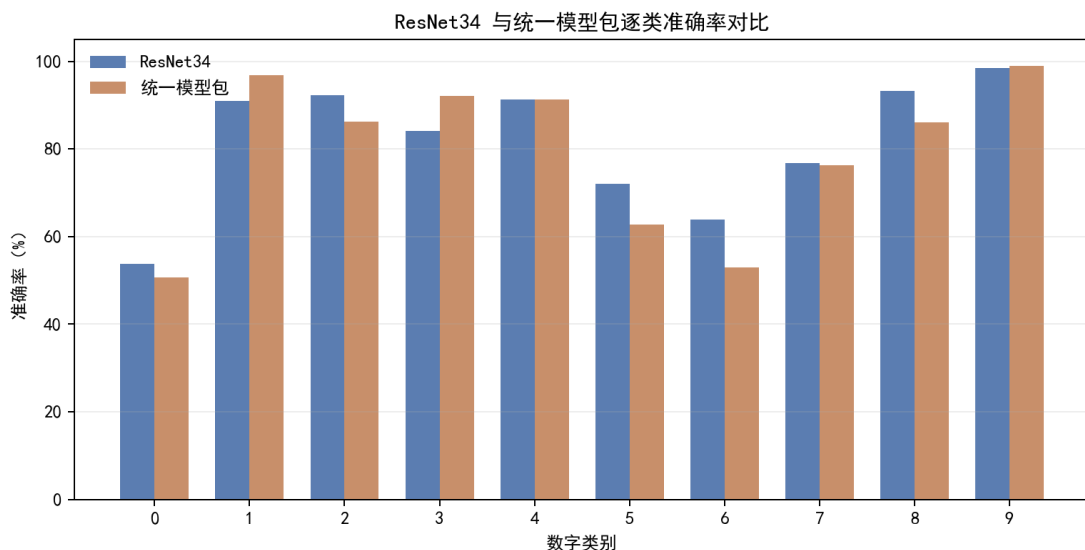


图 8 ResNet34 与 GestureDigitRecognizer_model.pt 逐类准确率对比

进一步观察逐类结果可以发现，GestureDigitRecognizer_model.pt 在 1、3、9 等类别上优于 ResNet34，其中数字 3 从 84.16% 提升到 92.08%，数字 1 从 91.02% 提升到 96.76%，数字 9 从 98.41% 提升到 99.01%。同时，部分数字如 2、5、6、8 因为集成权重和混淆校正的影响略有下降。因此本项目最终选择 GestureDigitRecognizer_model.pt 作为 Web 默认模型，是从综合使用体验出发，而不是只追求单一测试集整体准确率。

models Directory		
Name	Length	LastWriteTime
-----	-----	-----
chinese_cnn_0_9_aug.pkl	1,714,691	2026/6/30 11:48
chinese_efficientnet_b0_grouped_0_9.pth	16,359,535	2026/6/30 12:08
chinese_resnet34_grouped_0_9.pth	85,292,747	2026/6/30 12:23
GestureDigitRecognizer_model.json	1,994	2026/6/30 15:23
GestureDigitRecognizer_model.pt	161,024,811	2026/6/30 15:23
model_0-5test_own.pkl	1,715,583	2026/6/29 17:03
model_chinesetest_own.pkl	1,715,735	2026/6/29 16:44

图 9 models 文件夹中的模型文件

3.4 补充侧视角测试

考虑到摄像头使用时手势并不总是正对镜头，报告中补充了侧视角与斜视角测试。测试样例由新增测试集中的真实图片经过透视倾斜、旋转、缩放和背景弱化生成，用于模拟手部相对摄像头发生角度变化的情况。该组图片不是训练样本扩充，也不参与模型训练，只作为 GestureDigitRecognizer_model.pt 的补充可视化测试。



图 10 GestureDigitRecognizer_model.pt 侧视角补充测试样例

从图 10 可以看到，6 张侧视或斜视角增强样例均预测正确。该结果不能替代完整测试集准确率，但可以说明最终模型包在一定视角扰动下仍具有可用性，也展示了 ROI 和多模型集成对实际交互场景的补充价值。

3.5 Web 运行结果

启动服务：

& C:\Users\Alne\AppData\Local\miniconda3\envs\CV\python.exe app_server.py

访问地址：

http://127.0.0.1:8000/web/

服务健康检查结果显示：

```
{
  "ok": true,
  "device": "cuda",
  "default_mode": "auto",
  "packaged_model": {
    "enabled": true,
    "name": "GestureDigitRecognizer_model",
    "display_name": "手势数字识别模型"
  }
}
```

```
Web Service Health Output

GET /health
{
  "ok": true,
  "device": "cuda",
  "updated_at": "2026-06-30 15:20",
  "packaged_model": {
    "enabled": true,
    "name": "GestureDigitRecognizer_model",
    "display_name": "Gesture Digit Recognition Model",
    "version": "2026.06.30-GestureDigitRecognizer",
    "path": "models\\GestureDigitRecognizer_model.pt"
  }
}
```

图 11 Web 服务健康检查输出

单张样例图片接口测试结果：

```
{
  "digit": 0,
  "confidence": 0.8883,
  "device": "cuda",
  "model_key": "gesture_digit_recognizer",
  "roi_candidates": 5
}
```

Web 页面可完成图片上传识别和摄像头实时识别。页面默认展示最终识别结果；调试模式可以展开各模型预测结果，便于观察模型在不同数字上的差异。



图 12 Web 手势数字识别系统初始界面

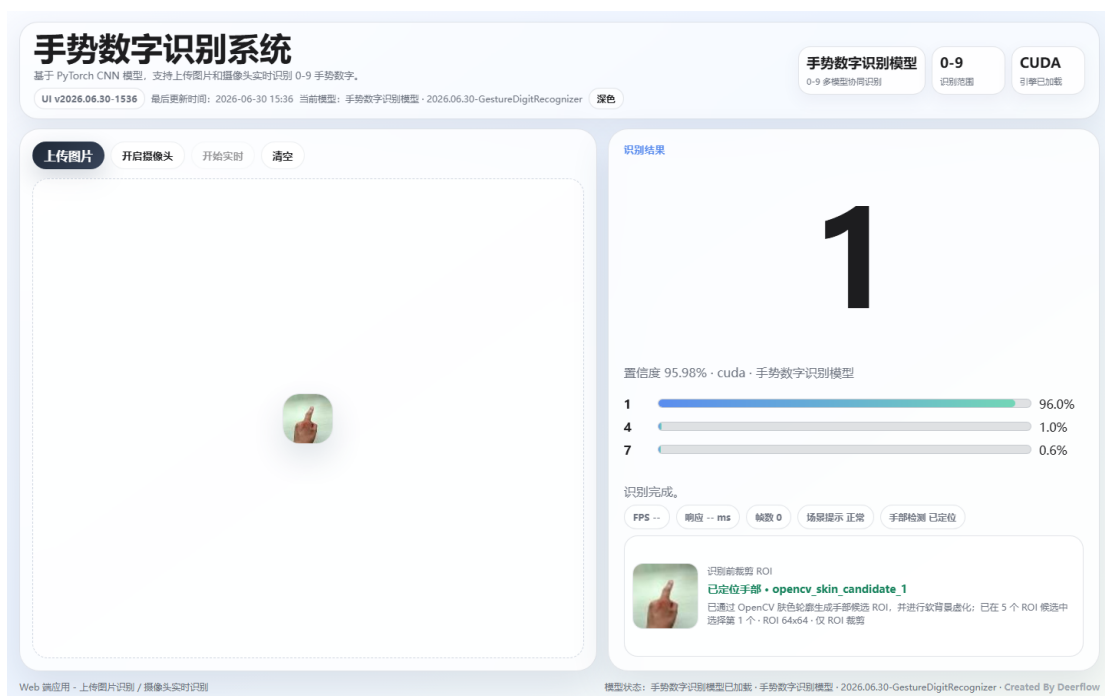


图 13 Web 端上传图片后的正确识别结果

为了避免报告中展示偶然样例，项目对测试集图片进行了接口预测，筛选出预测标签与真实标签一致的样本。下图展示的样例均为实际测试中预测正确的图片。

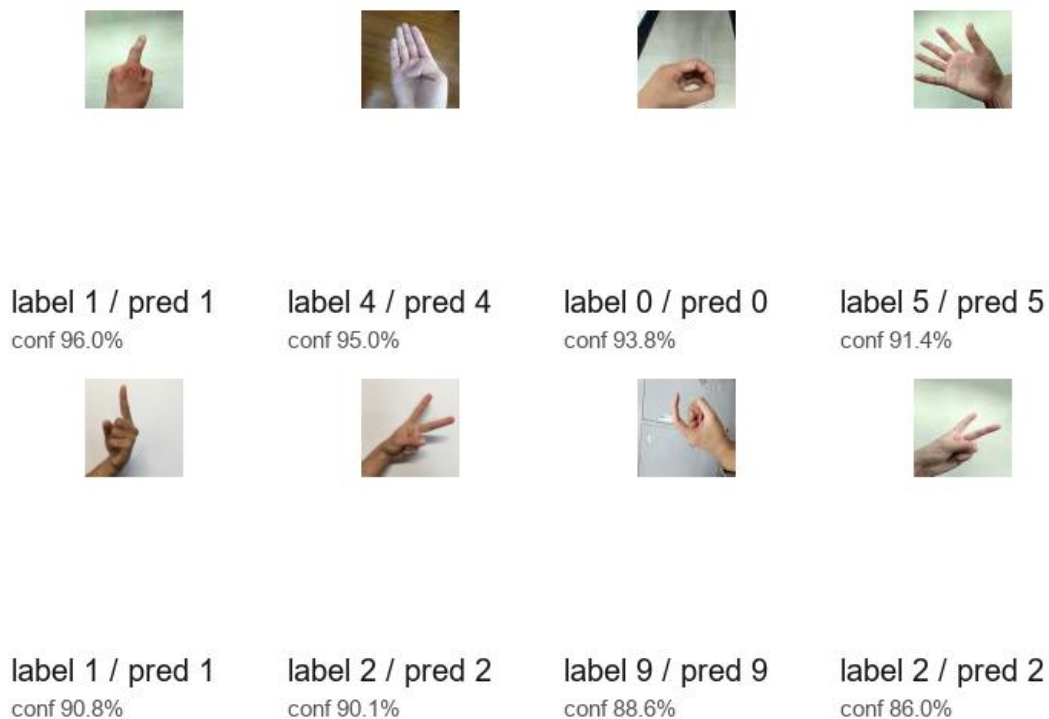


图 14 测试集中预测正确的部分样例

四、总结与体会

本次实训从指导书给出的初始代码出发，完成了深度学习图像分类项目的完整流程：数据读取、模型定义、训练、测试、推理和 Web 应用部署。基础 CNN 模型能够完成指导书要求，但在真实摄像头环境中表现不稳定，说明模型落地不能只依赖训练集准确率，还需要考虑数据分布、背景干扰、输入分辨率和实时预处理。

在扩展阶段，项目引入中国数字手势数据集，并按文件名前缀构建 0-9 标签。实验过程中发现，如果随机切分增强后的图片，同一来源手势的相似图片可能同时进入训练集和测试集，导致测试结果虚高。因此项目改为按来源组切分，这一点对评价模型泛化能力非常重要。原始指导书数据集承担了基础训练和代码复现任务，新增中国手势数据集承担了类别扩展和复杂场景优化任务，二者在项目中作用不同。

模型选择方面，增强 CNN 速度快但准确率不足；EfficientNet-B0 和 ResNet34 借助 ImageNet 预训练，在高分辨率输入下表现明显更好，其中 ResNet34 在分组测试中达到 84.19%。但不同模型擅长的数字不同，因此最终采用逐数字协同评分和易混类别校正，使 Web 端结果更贴近实际使用反馈。

在复杂场景优化方面，ROI 预处理是关键。直接把摄像头整帧输入模型会让背景干扰分类器，因此项目增加了手部候选区域提取、背景弱化和候选重排序。相比硬实例分割，软背景弱化更稳健，即使掩膜不完全准确，也不容易严重破坏手部主体。

通过本项目，我进一步理解了深度学习项目从“能训练”到“能使用”的差距。一个可落地的识别系统不仅需要模型本身，还需要可靠的数据划分、合理的数据增强、清晰的评估指标、输入预处理和用户界面。后续如果继续优化，可以进一步采集真实摄像头样本，针对 0、6 等低准确率类别补充数据，并尝试更专门的手部检测模型来提升 ROI 稳定性。